Data Quality & Pipeline Observability Platform

Project Proposal

Team: Mohammad Maher Metwaly, Sara Ahmed Abdelhamid, Malak Magdy Mohamed, Hager Abedlnasser Seddiq, Soltan Akram Soltan, Elhussieny Sabry Elshahat

Duration: 6 weeks

1. Project Overview

1.1 Problem Statement

A company operates several automated data pipelines, yet failures and bad data are discovered only after business users report incorrect dashboards. By then, faulty data has already reached decision-makers, and tracing the cause is slow and manual.

1.2 Proposed Solution

We propose a production-oriented ETL framework that validates data before it is loaded, records every failure, exposes operational metrics, and alerts the team when a run fails or behaves abnormally. Invalid records are isolated in an error store instead of reaching the target database, and every run leaves a traceable record of its outcome.

The project builds a representative ETL pipeline and enhances it with four capabilities:

- **Data-quality checks** across six rule families.
- **Pipeline reliability** through exception handling, retries, failed-task handling, dead-letter records, and execution logging.
- **Monitoring** of run duration, successful and failed runs, records processed, records rejected, and data-quality failures.
- **Observability and alerting** through Prometheus metrics, a Grafana dashboard, and alerts for failed or abnormal runs.

2. Objectives

1. **Detect data-quality problems early** by validating every batch against six rule families before loading.
2. **Protect the target database** by routing invalid records to an error/dead-letter store with timestamps, batch IDs, and rejection reasons.
3. **Make the pipeline reliable** by orchestrating it in Apache Airflow with retries for transient failures and clear FAILED status for permanent ones.
4. **Support safe reruns** so that re-running the same batch creates no duplicate records in the target.
5. **Record and expose operational metrics** so pipeline health is visible without waiting for user complaints.
6. **Alert proactively** on failed, stale, never-reported, and high-rejection pipeline runs.
7. **Demonstrate failure recovery** from an invalid batch, through detection and quarantine, to a successful rerun.

3. Scope

3.1 In Scope

Representative ETL pipeline

- Extraction from source files or database, including handling of missing files and empty batches.
- Transformation of valid records and idempotent loading into PostgreSQL.
- Processing counters for records extracted, processed, loaded, and rejected.

Data-quality checks

#	Rule family	Purpose
1	Null-value validation	Detect missing values in required fields
2	Duplicate detection	Identify repeated records by key or duplicate rule
3	Schema/type validation	Verify expected columns and data types
4	Row-count validation	Ensure the batch has an acceptable number of records
5	Referential-integrity checks	Confirm references exist in related data
6	Range validation	Detect values outside acceptable limits

Pipeline reliability

- Exception handling and retries with backoff.
- Failed-task handling, so permanent failures are never reported as successful runs.
- Dead-letter/error records for rejected data.
- Pipeline execution logging (run ID, status, timestamps, duration, row counts, error details) stored in PostgreSQL.

Monitoring and observability

- Metrics: pipeline duration, successful/failed runs, records processed, records rejected, and data-quality failures by rule.
- Prometheus metrics endpoint and scrape configuration.
- Grafana dashboard.
- Four alert scenarios: failed pipeline, stale pipeline, never-reported pipeline, and high data-rejection rate.

3.2 Deliverables

- Working ETL pipeline
- Airflow DAG
- Data-quality framework
- Error/rejection table or files
- Prometheus configuration
- Grafana dashboard
- Alert examples
- Failure-recovery demonstration
- Documentation (setup guide, architecture, and operational runbook)

3.3 Optional Extension

- Containerizing the complete environment with Docker Compose.
- Automated testing with GitHub Actions.

4. Architecture and Data Flow

1. Extract data from source files or database.
2. Run the data-quality validations.
3. Route invalid records to the error/dead-letter store.
4. Transform clean data and load it into the target database.
5. Emit metrics: records processed and rejected, duration, data-quality failures, and pipeline status.
6. Airflow orchestrates the Extract → Validate → Transform → Load → Quality Check sequence with retries and logging.
7. Prometheus scrapes the metrics; Grafana visualizes them and triggers alerts on failures or anomalies.

5. Technology Stack

Layer	Technology
Language	Python 3.10+
Orchestration	Apache Airflow
Data processing	Pandas
Database	MS SQL Server / T-SQL
Metrics	Prometheus
Visualization & alerting	Grafana
Containerization	Docker & Docker Compose
CI/CD	GitHub Actions
Logging	Python logging with structured logs

6. Team Structure

The work is divided into six roles, each owning a part of the deliverables.

Role	Focus	Primary deliverables
Data Pipeline Engineer	ETL pipeline and data processing	Working ETL pipeline, schemas, sample datasets
Data Quality Engineer	Validation rules and rejection management	Data-quality framework, error/rejection table or files
Orchestration Engineer	Airflow DAG and execution reliability	Airflow DAG, failure-recovery demonstration
Observability Engineer	Prometheus, Grafana, and alerts	Prometheus configuration, Grafana dashboard, alert examples
Test Engineer	Automated testing and recovery verification	Test suite and test report
DevOps Engineer	Environment, integration, and documentation	Integrated repository, documentation, Docker Compose, CI

PROJECT PLAN

Project Plan and 6-Week Execution Timeline

Author / Prepared by: El-Hussieny Sabry | Duration: 6 Weeks | Status: Detailed Baseline Plan

1. Purpose and Project Overview

The purpose of this document is to define the comprehensive execution plan, 6-week timeline, role abbreviations, detailed task schedule, dependencies, and milestone acceptance gates for the Data Quality & Pipeline Observability Platform.

Problem Statement & Objective: A company operates multiple automated data pipelines, but pipeline failures and corrupted data are currently discovered only after business users report inaccurate dashboards. This project builds a production-oriented ETL pipeline framework that proactively validates data quality across six rule families, isolates rejected records, handles transient and permanent failures in Apache Airflow, exposes operational metrics via Prometheus, visualizes pipeline health in Grafana, and triggers automated alerts.

This plan follows the exact structural methodology of the KPIs and Measurement Plan, linking all six engineering roles and 41 functional tasks (DE, DQ, OR, OB, QA, DO) directly to the 10 project KPIs (KPI-01 through KPI-10).

2. Team Structure, Role Abbreviations, and Member Roster

Each role is identified by a Role Number, a formal Role Title, and a two-letter Task Prefix / Abbreviation shown directly underneath the role name. This abbreviation is used across all task IDs, dependencies, and timeline tables. Team member names are left blank for assignment.

Role & Prefix	Role Title & Abbreviation Explanation	Assigned Member Name	Primary Ownership & Scope
Role 1 *Prefix: [DE]*	**Data Pipeline Engineer** *Abbreviation: DE (Data Engineering / ETL)*	Soltan Akram Soltan	Owns ETL pipeline (DE-01 to DE-06): Source extraction, transformations, idempotent target loading, processing counters, and valid/invalid sample datasets.
Role 2 *Prefix: [DQ]*	**Data Quality Engineer** *Abbreviation: DQ (Data Quality Validation)*	Mohammad Maher Metwaly, Hager Abedlnasser Seddiq	Owns validation framework (DQ-01 to DQ-07): Null-value, duplicate, schema/type, row-count, range, and referential-integrity checks, plus rejected-record storage.
Role 3 *Prefix: [OR]*	**Data Platform / Orchestration Engineer** *Abbreviation: OR (Orchestration & Reliability)*	Sara Ahmed Abdelhamid, Elhussieny Sabry Elshahat	Owns Airflow & reliability (OR-01 to OR-07): Airflow DAG, task sequencing, exception handling, retries, dead-letter routing, execution logging, and failure recovery.
Role 4 *Prefix: [OB]*	**Observability Engineer** *Abbreviation: OB (Observability & Monitoring)*	Malak Magdy Mohamed	Owns monitoring stack (OB-01 to OB-07): Prometheus metrics exposure, scraping configuration, Grafana dashboard panels, and operational alert rules.
Role 5 *Prefix: [QA]*	**Data Reliability / Test Engineer** *Abbreviation: QA (Quality Assurance & Testing)*	Elhussieny Sabry Elshahat, Sara Ahmed Abdelhamid	Owns verification & test evidence (QA-01 to QA-07): Unit tests for 6 DQ rules, end-to-end ETL tests, retry/failure verification, metric reconciliation, and test report.
Role 6 *Prefix: [DO]*	**DevOps Engineer** *Abbreviation: DO (DevOps, Deployment & Docs)*	Hager Abedlnasser Seddiq, Mohammad Maher Metwaly	Owns integration & delivery (DO-01 to QA-08): Repository layout, environment config, component integration, architecture & runbook docs, Docker Compose, and CI.

3. Project Phases and 6-Week Timeline Summary Table

The project is executed over six sequential weeks. The summary table below maps each weekly phase to its assigned Task IDs, Lead Roles (with abbreviations), Key Deliverables, and Linked KPIs.

Phase ID	Phase Name	Timeline	Included Task IDs	Lead Roles (Abbrev.)	Key Phase Deliverables	Linked KPIs
PH-01	Foundation, Architecture & Data Modeling	Week 1	DO-01, DO-03, DE-01, DE-06	**Role 6 (DO)** *Role 1 (DE)*	Repository structure, externalized config templates, target PostgreSQL schema, valid & invalid sample datasets.	KPI-09
PH-02	Core ETL & Data Quality Framework	Week 2	DE-02 to DE-05, DQ-01 to DQ-06	**Role 1 (DE)** *Role 2 (DQ)*	Repeatable ETL pipeline, processing counters, all 6 DQ validation rule modules, error/quarantine table or files.	KPI-01, KPI-04, KPI-05
PH-03	Orchestration, Reliability & Integration	Week 3	DQ-07, OR-01 to OR-06, DO-02	**Role 3 (OR)** *Role 2 (DQ), Role 6 (DO)*	Airflow DAG, task sequencing, retry & failed-task logic, dead-letter routing, and pipeline_runs execution logs.	KPI-03, KPI-06, KPI-07
PH-04	Observability, Metrics & Alerting Setup	Week 4	OB-01 to OB-07	**Role 4 (OB)** *Support: Role 3 (OR)*	Prometheus metrics endpoint, scraping config, Grafana dashboard, and 4 configured alert scenarios.	KPI-06, KPI-07, KPI-08
PH-05	Automated Testing, Recovery & CI/CD	Week 5	QA-01 to QA-06, OR-07, DO-06	**Role 5 (QA)** *Role 3 (OR), Role 6 (DO)*	Automated unit/integration test suite, failure-recovery demonstration, Docker Compose stack, GitHub Actions CI.	KPI-02, KPI-03, KPI-05, KPI-08
PH-06	Documentation, Runbook & Final Handover	Week 6	QA-07, QA-08, DO-04, DO-05	**Role 6 (DO)** *Role 5 (QA), All Roles*	Final test report, requirement traceability matrix, architecture documentation, and operational runbook.	KPI-09, KPI-10

4. Detailed Weekly Phase Execution Plan

This section provides the granular week-by-week execution breakdown, explaining how every role task is implemented, integrated, and verified against the project scope.

Phase 1 (PH-01): Foundation, Architecture & Data Modeling — Week 1

Lead Roles: Role 6 — DevOps Engineer [DO] & Role 1 — Data Pipeline Engineer [DE]

Objective: Establish the multi-folder repository layout, secure configuration standards, database schemas, and test datasets so all six roles can develop without blockers.

- **DO-01 — Define Project Repository Structure [Role 6 - DO]:** Create standardized directories for etl/, dags/, validation/, monitoring/, tests/, config/, and docs/ so every role has a clearly defined code location.
- **DO-03 — Manage Environment Configuration [Role 6 - DO]:** Externalize database credentials, file paths, and service ports using .env templates and configuration files, ensuring zero secrets are committed to Git.
- **DE-01 — Define Input Dataset and Target Data Model [Role 1 - DE]:** Specify at least one input dataset specification and document the target PostgreSQL schema (including primary keys, foreign keys, data types, and pipeline_runs metadata tables). Coordinate schema sign-off with Role 2 [DQ] and Role 3 [OR].
- **DE-06 — Prepare Sample Datasets for Demonstration [Role 1 - DE]:** Create at least one clean normal test batch (targeting <= 10% rejection rate for KPI-04) and one intentionally corrupted batch containing null values, duplicates, schema mismatches, out-of-range values, and broken foreign keys.
- **Week 1 Exit Criteria:** Repository structure merged, target schema signed off by DE, DQ, and OR, and sample datasets committed.

Phase 2 (PH-02): Core ETL & Data Quality Framework Development — Week 2

Lead Roles: Role 1 — Data Pipeline Engineer [DE] & Role 2 — Data Quality Engineer [DQ]

Objective: Build the repeatable ETL pipeline stages in parallel with the six modular data-quality validation rules and the rejected-records quarantine store.

- **DE-02 — Implement Data Extraction from Source [Role 1 - DE]:** Build extraction modules capable of reading source files/tables while gracefully handling missing files or zero-row empty batches.
- **DE-03 & DE-04 — Implement Transformations, Idempotent Loading, and End-to-End Flow [Role 1 - DE]:** Transform valid records and load them into PostgreSQL using idempotent upsert/deduplication logic so rerunning the same batch creates zero duplicate primary keys (KPI-05).
- **DE-05 — Add Processing Counters and Integration Hooks [Role 1 - DE]:** Instrument the ETL pipeline to expose exact counts for records extracted, inspected, processed, loaded, and rejected for downstream consumption by Role 3 [OR] and Role 4 [OB].
- **DQ-01 to DQ-05 — Implement All Six Data-Quality Rule Families [Role 2 - DQ]:** Build reusable validation checks covering: (1) Null-value validation on mandatory fields [DQ-01], (2) Duplicate key detection [DQ-02], (3) Schema and data-type validation [DQ-03], (4) Row-count expectations and numeric/date range bounds [DQ-04], and (5) Referential-integrity foreign key checks against reference data [DQ-05] (KPI-01).
- **DQ-06 — Implement Rejected-Record Quarantine Storage [Role 2 - DQ]:** Route invalid records to a dedicated error table or quarantine file along with timestamps, batch IDs, and specific rule failure reasons.
- **Week 2 Exit Criteria:** Complete end-to-end ETL run succeeds on valid input (DE-04), idempotency verified (KPI-05), and all 6 DQ rules execute and quarantine bad records accurately (KPI-01).

Phase 3 (PH-03): Orchestration, Reliability & Component Integration — Week 3

Lead Roles: Role 3 — Orchestration Engineer [OR], Role 2 — Data Quality Engineer [DQ], Role 6 — DevOps Engineer [DO]

Objective: Integrate the validation engine into the ETL workflow and orchestrate the entire pipeline inside an Apache Airflow DAG with full reliability, retry, and logging mechanisms.

- **DQ-07 — Integrate Validation into the ETL Flow [Role 2 - DQ]:** Embed all six validation categories into the pipeline execution flow so records are inspected before target loading and produce measurable pass/fail outcomes.
- **OR-01 & OR-02 — Create Airflow DAG and Configure Execution Order [Role 3 - OR]:** Define the Airflow DAG with explicit task dependencies (Extract -> Validate -> Quarantine/Transform -> Load -> Emit Metrics), ensuring validation blocks corrupted batches from polluting target tables.
- **OR-03 & OR-04 — Implement Exception Handling, Retries, and Failed-Task Handling [Role 3 - OR]:** Configure automated retry policies (e.g., retry count and backoff delay) for transient failures, while ensuring permanent validation or system failures mark the Airflow DAG run as FAILED rather than succeeding silently.
- **OR-05 & OR-06 — Implement Dead-Letter Routing and Execution Logging [Role 3 - OR]:** Connect failed/rejected records to the agreed dead-letter storage and record comprehensive run metadata (run_id, status, start/end timestamps, duration_seconds, row counters, and error summaries) in PostgreSQL (KPI-06, KPI-07).
- **DO-02 — Integrate Application Components [Role 6 - DO]:** Connect ETL, DQ, PostgreSQL, and Airflow components to run cohesively under a unified configuration.
- **Week 3 Exit Criteria:** Airflow DAG orchestrates all stages, logs run outcomes and durations in PostgreSQL (KPI-06, KPI-07), and isolates rejected records within the <= 10% normal threshold (KPI-04).

Phase 4 (PH-04): Observability, Metrics & Alerting Setup — Week 4

Lead Roles: Role 4 — Observability Engineer [OB] (Supported by Role 1 [DE], Role 3 [OR], Role 6 [DO])

Objective: Expose real-time pipeline execution and data-quality metrics to Prometheus, build the operational Grafana dashboard, and configure automated alerting.

- **OB-01 to OB-04 — Expose Operational and Data-Quality Metrics [Role 4 - OB]:** Expose Prometheus metrics for batch execution duration (OB-01), successful vs. failed run counters/statuses (OB-02), records processed and rejected (OB-03), and validation failures broken down by rule category (OB-04).
- **OB-05 — Configure Prometheus Scraping [Role 4 - OB]:** Configure prometheus.yml to scrape the application metrics endpoint or exporter at defined intervals.
- **OB-06 — Build the Grafana Dashboard [Role 4 - OB]:** Design a comprehensive Grafana dashboard displaying latest run outcome (KPI-07), batch execution duration gauge/trends (KPI-06), processed vs. rejected record counts, rejection rate percentage (KPI-04), and DQ rule failure breakdowns.
- **OB-07 — Configure Failure and Abnormal-Run Alerts [Role 4 - OB]:** Implement and demonstrate all four required alert scenarios defined in KPI-08: (1) Failed Pipeline Run, (2) Stale Pipeline (e.g., >26 hours since last success), (3) Never-Reported Pipeline (missing metrics), and (4) High Data-Rejection Rate (>10%).
- **Week 4 Exit Criteria:** Prometheus actively scrapes all metrics, Grafana panels render live run data, and all 4 alert rules are configured (KPI-08).

Phase 5 (PH-05): Automated Testing, Failure Recovery & Containerization — Week 5

Lead Roles: Role 5 — Test Engineer [QA], Role 3 — Orchestration Engineer [OR], Role 6 — DevOps Engineer [DO]

Objective: Execute formal unit, integration, and reliability test suites, demonstrate end-to-end failure recovery, and containerize the platform with CI automation.

- **QA-01 — Create Unit Tests for Data-Quality Rules [Role 5 - QA]:** Implement automated pytest unit tests for all six validation categories, ensuring every category has at least 1 passing test and 1 failing test (KPI-01, KPI-02).
- **QA-02 & QA-03 — Test Complete Valid and Invalid ETL Flows [Role 5 - QA]:** Verify that valid batches load cleanly without duplicates (KPI-05) and invalid batches quarantine bad rows without leaking corrupted data into the target table.
- **QA-04, QA-05 & QA-06 — Verify Retries, Metric Accuracy, Dashboards, and Alerts [Role 5 - QA]:** Test transient retry and permanent failure scenarios (QA-04), verify that Prometheus/Grafana numbers match PostgreSQL records exactly (QA-05), and trigger all four alert conditions to confirm firing behavior (QA-06, KPI-08).
- **OR-07 — Demonstrate Failure Recovery [Role 3 - OR, Supported by OB & QA]:** Execute the formal KPI-03 recovery scenario: submit an invalid batch -> verify FAILED run and quarantined records -> fix the underlying data issue -> rerun the pipeline -> verify SUCCESS status and updated dashboard metrics.
- **DO-06 & QA-08 — Containerize Environment & Add Automated CI Testing**
- **[Role 6 – DO, Role 5-QA]:** Package PostgreSQL, Airflow, ETL/DQ modules, Prometheus, and Grafana in Docker Compose with health checks (DO-06), and configure GitHub Actions to run the automated test suite on every push or pull request (QA-08).
- **Week 5 Exit Criteria:** 100% automated test pass rate achieved (KPI-02), failure-recovery demonstration recorded (KPI-03), and Docker Compose + GitHub Actions running cleanly.

Phase 6 (PH-06): Documentation, Runbook & Final Delivery — Week 6

Lead Roles: Role 6 — DevOps Engineer [DO], Role 5 — Test Engineer [QA], All Roles

Objective: Finalize the test report, requirement traceability matrix, architecture documentation, and operational runbook for final submission and demonstration.

- **QA-07 — Produce Final Test Report & Traceability Matrix [Role 5 - QA]:** Document all executed test cases, expected vs. actual results, defect resolutions, and complete the Functional Requirement Traceability Matrix linking >= 90% of requirements (and 100% of critical requirements) to code modules and tests (KPI-09).
- **DO-04 — Document Installation and Execution [Role 6 - DO]:** Write step-by-step clean-environment setup instructions, dependency configurations, and manual pipeline trigger commands.
- **DO-05 — Document Architecture and Operational Runbook [Role 6 - DO]:** Complete the architecture guide and operational runbook detailing data flow, validation rules, retry behavior, metric definitions, alert thresholds, log inspection, and step-by-step failure-recovery instructions (KPI-10).
- **Week 6 Exit Criteria:** All 10 KPIs marked as Passed with verifiable evidence, operational runbook validated by reviewer walkthrough (KPI-10), and final project package ready for presentation.

5. Complete 41-Task Master Schedule by Role (Task-to-Week & KPI Mapping)

To ensure 100% alignment with the Task Assignment & Roles document and the KPIs Measurement Plan, the master table below schedules all 41 individual tasks across the 6 weeks.

Task ID	Role (Abbrev.)	Defined Task Summary	Scheduled Week	Measurable Acceptance Criteria & Linked KPI
DE-01	Role 1 [DE]	Define input dataset and target data model.	Week 1 (PH-01)	At least 1 input dataset and 1 documented target schema (KPI-09).
DE-02	Role 1 [DE]	Implement data extraction from source.	Week 2 (PH-02)	Pipeline reads source and handles empty/missing input appropriately.
DE-03	Role 1 [DE]	Implement transformations and data loading.	Week 2 (PH-02)	Valid records reach target with zero duplicate primary keys on rerun (KPI-05).
DE-04	Role 1 [DE]	Integrate ETL stages into a repeatable pipeline.	Week 2 (PH-02)	At least 1 complete end-to-end run succeeds on valid input (KPI-05).
DE-05	Role 1 [DE]	Add processing counters and integration hooks.	Week 2 (PH-02)	Extracted, processed, loaded, and rejected counts collected downstream (KPI-04).
DE-06	Role 1 [DE]	Prepare sample datasets for demonstration.	Week 1 (PH-01)	At least 1 valid dataset (<=10% rejection) and 1 invalid dataset (KPI-03, KPI-04).
DQ-01	Role 2 [DQ]	Implement null-value validation.	Week 2 (PH-02)	Detects missing values in designated mandatory fields (KPI-01).
DQ-02	Role 2 [DQ]	Implement duplicate detection.	Week 2 (PH-02)	Identifies repeated records using defined key or duplicate rule (KPI-01).
DQ-03	Role 2 [DQ]	Implement schema and type validation.	Week 2 (PH-02)	Rejects records with missing required fields or incompatible types (KPI-01).
DQ-04	Role 2 [DQ]	Implement row-count and range validation.	Week 2 (PH-02)	Checks configured row-count bounds and field min/max limits (KPI-01).
DQ-05	Role 2 [DQ]	Implement referential-integrity checks.	Week 2 (PH-02)	Detects records whose foreign keys do not exist in reference data (KPI-01).
DQ-06	Role 2 [DQ]	Implement rejected-record storage.	Week 2 (PH-02)	Invalid records written to error table/files with rejection reasons (KPI-03, KPI-04).

Task ID	Role (Abbrev.)	Defined Task Summary	Scheduled Week	Measurable Acceptance Criteria & Linked KPI
DQ-07	Role 2 [DQ]	Integrate validation into the ETL flow.	Week 3 (PH-03)	All 6 validation categories execute and produce pass/fail results (KPI-01).
OR-01	Role 3 [OR]	Create the Airflow DAG.	Week 3 (PH-03)	DAG contains required ETL and DQ stages and their dependencies.
OR-02	Role 3 [OR]	Configure task execution order.	Week 3 (PH-03)	Tasks execute in sequence, validating before loading target data.
OR-03	Role 3 [OR]	Implement exception handling and retries.	Week 3 (PH-03)	A transient failure triggers the configured retry policy.
OR-04	Role 3 [OR]	Implement failed-task handling.	Week 3 (PH-03)	Permanent failure is visible in Airflow and marked as FAILED (KPI-03, KPI-07).
OR-05	Role 3 [OR]	Implement dead-letter/error handling.	Week 3 (PH-03)	Failed or rejected records routed to agreed error-storage mechanism (KPI-03).
OR-06	Role 3 [OR]	Add pipeline execution logging.	Week 3 (PH-03)	Each run records run ID, status, duration_seconds, and errors (KPI-06, KPI-07).
OR-07	Role 3 [OR]	Demonstrate failure recovery.	Week 5 (PH-05)	Induced failure and subsequent recovery rerun are recorded and reproducible (KPI-03).
OB-01	Role 4 [OB]	Expose pipeline duration metrics.	Week 4 (PH-04)	Each completed run contributes a measurable duration value (<=60s target, KPI-06).
OB-02	Role 4 [OB]	Expose successful and failed run metrics.	Week 4 (PH-04)	Run outcomes (SUCCESS/FAILED) reflected in queryable metrics (KPI-07).
OB-03	Role 4 [OB]	Expose processing and rejection metrics.	Week 4 (PH-04)	Records processed and records rejected are measurable (KPI-04).
OB-04	Role 4 [OB]	Expose data-quality failure metrics.	Week 4 (PH-04)	Validation failures can be inspected by rule or failure category (KPI-01).
OB-05	Role 4 [OB]	Configure Prometheus scraping.	Week 4 (PH-04)	Prometheus successfully scrapes the application metrics endpoint.
OB-06	Role 4 [OB]	Build the Grafana dashboard.	Week 4 (PH-04)	Dashboard displays duration, run status, row counts, rejections, and DQ failures (KPI-07).
OB-07	Role 4 [OB]	Configure failure and abnormal-run alerts.	Week 4 (PH-04)	All 4 alert scenarios (Failed, Stale, Never-Reported, High Rejection) configured (KPI-08).
QA-01	Role 5 [QA]	Create unit tests for data-quality rules.	Week 5 (PH-05)	Every validation category has >= 1 passing and >= 1 failing test (KPI-01, KPI-02).
QA-02	Role 5 [QA]	Test the complete ETL flow.	Week 5 (PH-05)	At least 1 valid-data integration test completes with zero duplicates (KPI-02, KPI-05).
QA-03	Role 5 [QA]	Test invalid-data handling.	Week 5 (PH-05)	Invalid records are quarantined without being loaded as valid (KPI-02, KPI-03).
QA-04	Role 5 [QA]	Test retry and permanent-failure scenarios.	Week 5 (PH-05)	At least 1 transient-failure and 1 permanent-failure scenario verified (KPI-02).
QA-05	Role 5 [QA]	Verify metric accuracy.	Week 5 (PH-05)	Reported processed, rejected, and run status counts match DB evidence (KPI-07).
QA-06	Role 5 [QA]	Test dashboards and alerts.	Week 5 (PH-05)	All 4 configured alerts trigger under intended conditions (KPI-08).
QA-07	Role 5 [QA]	Produce a test report & traceability matrix.	Week 6 (PH-06)	Test results documented and >=90% requirements mapped to tests (KPI-02, KPI-09).
DO-01	Role 6 [DO]	Define project repository structure.	Week 1 (PH-01)	ETL, DAGs, validation, monitoring, tests, config, and docs have clear folders.

Task ID	Role (Abbrev.)	Defined Task Summary	Scheduled Week	Measurable Acceptance Criteria & Linked KPI
DO-02	Role 6 [DO]	Integrate application components.	Week 3 (PH-03)	Agreed components run together using a documented configuration.
DO-03	Role 6 [DO]	Manage environment configuration.	Week 1 (PH-01)	Settings and secrets externalized; no real credentials committed.
DO-04	Role 6 [DO]	Document installation and execution.	Week 6 (PH-06)	Reviewer can follow setup instructions from a clean environment (KPI-10).
DO-05	Role 6 [DO]	Document architecture and operations runbook.	Week 6 (PH-06)	Docs explain data flow, DQ rules, retries, metrics, alerts, and recovery (KPI-10).
DO-06	Role 6 [DO]	Containerize the environment (Optional).	Week 5 (PH-05)	Docker Compose starts configured services and health checks pass.
QA-08	Role 6 [DO]	Add automated CI testing (Optional).	Week 5 (PH-05)	GitHub Actions runs configured tests on PR/push and reports pass/fail (KPI-02).

6. Cross-Role Dependencies and Handoff Matrix

To prevent schedule delays across the six roles, the following critical handoffs must be completed on time:

Downstream Task	Receiving Role	Prerequisite Task	Providing Role	Required Handoff & Interface Specification
DQ-01 to DQ-06	Role 2 [DQ]	DE-01, DE-06	Role 1 [DE]	Role 1 supplies finalized column names, data types, reference keys, and sample datasets so Role 2 can code validation rules.
DQ-07	Role 2 [DQ]	DE-02, DE-03	Role 1 [DE]	Role 1 provides extraction and transformation data structures so Role 2 can hook validation before target loading.
OR-01 to OR-06	Role 3 [OR]	DE-04, DE-05, DQ-07	Role 1 [DE], Role 2 [DQ]	Roles 1 and 2 provide callable pipeline functions, return codes, and row counters for Airflow DAG orchestration.
OB-01 to OB-07	Role 4 [OB]	DE-05, OR-06	Role 1 [DE], Role 3 [OR]	Roles 1 and 3 expose execution timestamps, run statuses, and row/error counts for Prometheus scraping and Grafana panels.
QA-01 to QA-06	Role 5 [QA]	All DE, DQ, OR, OB Tasks	Roles 1 to 4	Implementation roles deliver testable modules, DAGs, metrics, and alert rules for automated QA verification.
DO-04 to QA-08	Role 6 [DO]	All Implementation Tasks	Roles 1 to 5	All roles provide service port mappings, environment variables, and operational steps for Docker Compose and the Runbook.

7. Milestone Gates and KPI Evaluation Schedule

In accordance with Section 5 of the KPIs and Measurement Plan, each KPI is evaluated at its designated project stage and marked as Not Tested, Passed, or Failed based on verifiable evidence:

Milestone ID	Evaluation Stage	Milestone Gate Name	Target KPIs & Verifiable Acceptance Criteria
MS-01	End of Week 2 (Development)	Core ETL & DQ Rule Freeze	KPI-01 (6/6 DQ rule families implemented) & KPI-05 (Zero duplicate primary keys on batch rerun).
MS-02	End of Week 3 (Integration)	Orchestration & Quarantine Freeze	KPI-04 (Normal batch rejection rate <= 10%) & KPI-07 (Every run records SUCCESS/FAILED outcome in PostgreSQL).
MS-03	End of Week 4 (Monitoring Setup)	Observability & Alerting Freeze	KPI-06 (Batch execution duration baselined at <= 60s) & KPI-08 (All 4 alert scenarios configured in Grafana/Prometheus).
MS-04	End of Week 5 (System Testing)	Reliability & Recovery Gate	KPI-02 (100% required automated tests pass) & KPI-03 (Corrupted batch fails & quarantines; corrected batch recovers to SUCCESS).
MS-05	End of Week 6 (Release Review)	Final Handover & Submission Gate	KPI-09 (>= 90% functional requirement traceability with 100% critical coverage) & KPI-10 (Operational runbook complete and verified).

8. Risk Management and Mitigation Plan

The table below identifies key technical and schedule risks across the 6-week timeline and defines proactive mitigation steps:

Risk ID	Risk Description	Impact	Mitigation Strategy & Responsible Roles
RSK-01	Delayed schema definition (DE-01) blocks DQ rules (Role 2 [DQ]) and Airflow logging (Role 3 [OR]).	High	Enforce a formal Schema Freeze by Day 3 of Week 1 before writing downstream transformation or validation code (Role 1 [DE], Role 2 [DQ]).
RSK-02	Environment inconsistencies across team laptops cause integration failures in Week 3 and Week 5.	Medium	Establish standardized .env templates (DO-03) and initial Docker Compose service definitions in Week 1 (Role 6 [DO]).
RSK-03	Mismatch between PostgreSQL run tables and Prometheus metrics causes QA-05 failures.	High	Emit both database logs and Prometheus gauges from the same centralized execution counters in DE-05 and OR-06 (Role 1 [DE], Role 3 [OR], Role 4 [OB]).
RSK-04	Batch execution duration exceeds the 60-second provisional target on demonstration runs (KPI-06).	Low	Fix the demonstration dataset size early in Week 1 (DE-06) and use batch/bulk database inserts during loading (Role 1 [DE], Role 5 [QA]).

9. Conclusion

This Project Plan and 6-Week Execution Timeline provides a complete, structured roadmap for delivering the Data Quality & Pipeline Observability Platform. By clearly defining role abbreviations (DE, DQ, OR, OB, QA, DO), providing open roster slots for team member names, scheduling all 41 tasks across six weekly phases, and linking every milestone to the 10 project KPIs, the team can execute smoothly, verify quality at every gate, and deliver a production-ready platform on schedule.

Project Deliverables and Resource Allocation Plan

1. Deliverables Summary Table

This table outlines the core deliverables required for the successful completion, verification, and release acceptance of the platform, mapped against corresponding project KPIs and assigned team members:

Deliverable ID	Deliverable Name	Definition / Description & Ownership	Aligned KPIs
DEL-01	Working ETL Pipeline	Production-oriented ETL framework executing data extraction, transformation, and loading. (Owner: Soltan Akram Soltan / DE)	KPI-06
DEL-02	Data Quality Framework	Implementation and testing of all six required data-quality rule families. (Owner: Mohammad Maher Metwaly / DQ-DO)	KPI-01, KPI-04
DEL-03	Rejection & Quarantine Mechanism	Systematic detection and isolation of invalid input rows into a designated quarantine store. (Owner: Mohammad Maher Metwaly / DQ-DO)	KPI-03, KPI-04
DEL-04	Idempotent Data Loading	Ensuring pipeline rerun safety and verifying zero unintended duplicate primary-key records.	KPI-05
DEL-05	Orchestration & Airflow DAGs	Task scheduling and execution workflows handling automated retries and failure detection. (Owner: Sara Ahmed / Elhussieny Sabry)	KPI-03, KPI-06
DEL-06	Observability Dashboard	Prometheus metrics collection and Grafana dashboard exposing real-time run outcomes and metrics. (Owner: Malak Magdy Mohamed / OB)	KPI-06, KPI-07
DEL-07	Alerting System	Configured alert rules and testing for critical scenarios: Failed pipeline, Stale pipeline, Never-reported pipeline. (Owner: Malak Magdy Mohamed / OB)	KPI-08
DEL-08	Automated Test Suite	Automated Python test cases achieving target test pass rates, accompanied by Traceability Matrix. (Owner: Elhussieny Sabry / Sara Ahmed)	KPI-02, KPI-09
DEL-09	Operational Runbook	Complete documentation covering environment setup, execution instructions, and troubleshooting. (Owner: Hager Abedlnasser / Mohammad Maher)	KPI-10

2. Resource Allocation Summary Table

This table defines the breakdown of roles, assigned team members, core responsibilities, and primary deliverables based on the project scope:

Role ID	Role Title & Member	Core Focus & Responsibilities	Primary Deliverables
ROLE-01	Data Pipeline Engineer Soltan Akram Soltan (DE)	ETL pipeline and data processing. Builds core ETL data flow including extraction, transformation, and loading.	Working ETL Pipeline
ROLE-02	Data Quality Engineer Mohammad Maher Metwaly (DQ, DO)	Validation rules and rejection management. Implements all six data-quality checks and rejected-records mechanism.	Data Quality & Quarantine Store
ROLE-03	Orchestration Engineer Sara Ahmed Abdelhamid (OR, QA)	Airflow DAG and execution reliability. Orchestrates tasks, handles retries and failures, and records execution logs.	Airflow DAGs & Orchestration
ROLE-04	Observability Engineer Malak Magdy Mohamed (OB)	Prometheus, Grafana, metrics, and alerts. Exposes operational metrics, creates monitoring dashboards, and configures alerts.	Grafana Dashboard & Alerts
ROLE-05	Test & Reliability Engineer Elhussieny Sabry Elshahat (QA, OR)	Automated testing and failure-recovery verification. Tests data-quality rules, pipeline reliability, and metric accuracy.	Test Suite & Traceability
ROLE-06	DevOps Engineer Hager Abedlnasser Seddiq (DO, DQ)	Environment, deployment, integration, and documentation. Integrates environment, prepares run instructions, and operational runbook.	Operational Runbook & Setup

TASK ASSIGNMENT & ROLES

Six-Role Work Breakdown and Deliverable Ownership

Name	Role	Abbreviation
Mohammad Maher Metwaly	Data Quality Engineer, DevOps Engineer (Team Leader)	DQ - DO
Hager Abedlnasser Seddiq	DevOps Engineer, Data Quality Engineer	DO - DQ
Elhussieny Sabry Elshahat	Data Reliability /Test Engineer, Data Platform / Orchestration Engineer	QA - OR
Sara Ahmed Abdelhamid	Data Platform / Orchestration Engineer, Data Reliability /Test Engineer	OR - QA
Soltan Akram Soltan	Data Pipeline Engineer	DE
Malak Magdy Mohamed	Observability Engineer	OB

Project Overview

The project brief defines a production-oriented ETL framework that detects data-quality issues, handles pipeline failures, records operational metrics, and provides monitoring. Required deliverables include a working ETL pipeline, Airflow DAG, data-quality framework, error/rejection records, Prometheus configuration, Grafana dashboard, alert examples, recovery demonstration, and documentation. Docker Compose and GitHub Actions are optional extensions.

The six roles and acceptance criteria below are assignments based on the project scope.

The task quantities and thresholds are project targets.

1. Team Structure

Role 1 — Data Pipeline Engineer

Focus: ETL pipeline and data processing. Builds the ETL pipeline, including extraction, transformation, loading, and data flow.

Role 2 — Data Quality Engineer

Focus: Validation rules and rejection management. Implements all six data-quality checks and creates the rejected-records mechanism.

Role 3 — Data Platform / Orchestration Engineer

Focus: Airflow DAG and execution reliability. Orchestrates tasks, handles retries and failures, and records pipeline execution logs.

Role 4 — Observability Engineer

Focus: Prometheus, Grafana, metrics, and alerts. Exposes operational metrics, creates the monitoring dashboard, and configures alerts.

Role 5 — Data Reliability / Test Engineer

Focus: Automated testing and failure-recovery verification. Tests data-quality rules, pipeline reliability, metric accuracy, and recovery scenarios.

Role 6 — DevOps Engineer

Focus: Environment, deployment, integration, and documentation. Integrates the environment, prepares run instructions, and owns optional Docker Compose and GitHub Actions work.

These are proposed job titles and task assignments based on the project scope, rather than roles specified in the PDF.

2. Detailed Task Allocation by Role

Role 1 — Data Pipeline Engineer

Owns: ETL pipeline

Task ID	Defined task	Measurable acceptance criteria
DE-01	Define the input dataset and target data model.	At least 1 input dataset and 1 documented target schema.
DE-02	Implement data extraction from the source.	Pipeline reads the source and handles empty or missing input appropriately.
DE-03	Implement transformations and data loading.	Valid records reach the target without unintended duplication.
DE-04	Integrate the ETL stages into a repeatable pipeline.	At least 1 complete end-to-end run succeeds on valid input.
DE-05	Add processing counters and integration hooks.	Extracted, processed, loaded, and rejected counts can be collected downstream.
DE-06	Prepare sample datasets for demonstration.	At least 1 valid dataset and 1 dataset containing intentionally invalid records.

Deliverables: ETL source code, input and target schemas, sample datasets, and a working end-to-end data flow.

Dependencies: Coordinate the target schema with Role 2 and task interfaces with Role 3.

Role 2 — Data Quality Engineer

Owns: validation framework

Task ID	Defined task	Measurable acceptance criteria
DQ-01	Implement null-value validation.	Detects missing values in designated mandatory fields.
DQ-02	Implement duplicate detection.	Identifies repeated records using a defined key or duplicate rule.
DQ-03	Implement schema and type validation.	Rejects records with missing required fields or incompatible data types.
DQ-04	Implement row-count and range validation.	Checks configured row-count expectations and field-specific minimum/maximum bounds.
DQ-05	Implement referential-integrity checks.	Detects records whose reference keys do not exist in the reference dataset.
DQ-06	Implement rejected-record storage.	Invalid records are written to an error table or files with rejection reasons.
DQ-07	Integrate validation into the ETL flow.	All six validation categories execute and produce measurable pass/fail results.

Deliverables: Reusable data-quality framework, validation rules, rejection table or files, and validation results.

Dependencies: Role 1 supplies schema and input data. Role 3 determines how validation failures affect task execution.

Role 3 — Data Platform / Orchestration Engineer

Owns: Airflow and pipeline reliability

Task ID	Defined task	Measurable acceptance criteria
OR-01	Create the Airflow DAG.	DAG contains the required ETL stages and their dependencies.
OR-02	Configure task execution order.	Tasks execute in the correct sequence, with validation before inappropriate data is loaded.
OR-03	Implement exception handling and retries.	A transient failure triggers the configured retry policy.
OR-04	Implement failed-task handling.	A permanent task failure is visible in Airflow and is not marked as a successful run.
OR-05	Implement dead-letter/error handling.	Failed or rejected records are routed to the agreed error-storage mechanism.
OR-06	Add execution logging.	Each run records its run identifier, status, timestamps, and relevant error details.
OR-07	Demonstrate failure recovery.	A deliberately induced failure and subsequent successful recovery are recorded and reproducible.

Deliverables: Airflow DAG, task configuration, retry and failure-handling logic, execution logs, and recovery demonstration.

Dependencies: Roles 1 and 2 provide pipeline stages and validation interfaces. Role 4 consumes execution events and metrics.

Role 4 — Observability Engineer

Owns: Prometheus, Grafana, and alerts

Task ID	Defined task	Measurable acceptance criteria
OB-01	Expose pipeline duration metrics.	Each completed run contributes a measurable duration value.
OB-02	Expose successful and failed run metrics.	Run outcomes are reflected in queryable metrics.
OB-03	Expose processing and rejection metrics.	Records processed and records rejected are measurable.
OB-04	Expose data-quality failure metrics.	Validation failures can be inspected by rule or failure category.
OB-05	Configure Prometheus scraping.	Prometheus successfully scrapes the application metrics endpoint.
OB-06	Build the Grafana dashboard.	Dashboard displays duration, run status, record counts, rejection counts, and data-quality failures.
OB-07	Configure failure and abnormal-run alerts.	At least 1 failed-run alert and 1 abnormal-run alert can be demonstrated.

Deliverables: Prometheus metrics endpoint or exporter integration, Prometheus configuration, Grafana dashboard, alert rules, and alert examples.

Dependencies: Roles 1 and 3 provide metric sources and run metadata. Role 5 verifies metric correctness and alert behavior.

Role 5 — Data Reliability / Test Engineer

Owns: verification and test evidence

Task ID	Defined task	Measurable acceptance criteria
QA-01	Create unit tests for data-quality rules.	Every validation category has at least 1 passing test and 1 failing test.
QA-02	Test the complete ETL flow.	At least 1 valid-data integration test completes successfully.
QA-03	Test invalid-data handling.	Invalid records are rejected or quarantined according to agreed rules, without being incorrectly loaded as valid.
QA-04	Test retry and permanent-failure scenarios.	At least 1 transient-failure scenario and 1 permanent-failure scenario are verified.
QA-05	Verify metric accuracy.	Reported processed, rejected, successful-run, and failed-run counts match test evidence.
QA-06	Test dashboards and alerts.	Each configured alert triggers under its intended condition, and expected monitoring information is visible.
QA-07	Produce a test report.	Test cases, expected results, actual results, and unresolved defects are documented.
QA-08	Add automated CI testing — optional.	GitHub Actions runs configured tests on a pull request or push and reports pass/fail status.

Deliverables: Automated test suite, failure-injection scenarios, test report, and evidence of successful recovery and alerting.

Dependencies: All implementation roles provide testable components. Role 5 verifies integration rather than reimplementing components.

Role 6 — DevOps Engineer

Owns: integration, environment, and delivery

Task ID	Defined task	Measurable acceptance criteria
DO-01	Define the project repository structure.	ETL, DAGs, validation, monitoring, tests, configuration, and documentation have clearly defined locations.
DO-02	Integrate the application components.	The agreed components run together using a documented configuration.
DO-03	Manage environment configuration.	Required settings and secrets are externalized; no real credentials are committed to the repository.
DO-04	Document installation and execution.	A reviewer can follow setup instructions from a clean environment and run the demonstrated workflow.
DO-05	Document architecture and operations.	Documentation explains data flow, validation rules, retry behavior, metrics, alerts, and recovery.
DO-06	Containerize the environment — optional.	Docker Compose starts the configured services and documented health checks pass.

Deliverables: Integrated repository, configuration templates, setup guide, architecture documentation, operational runbook, and optional Docker Compose and GitHub Actions files.

Dependencies: All five roles provide components and configuration requirements. Role 6 integrates final deliverables without replacing implementation ownership.

3. Deliverable Ownership Matrix

Deliverable	Primary owner	Supporting roles
Working ETL pipeline	Role 1 — Data Pipeline Engineer	Roles 2, 3, 5
Airflow DAG	Role 3 — Orchestration Engineer	Roles 1, 2
Data-quality framework	Role 2 — Data Quality Engineer	Roles 1, 5
Error/rejection table or files	Role 2 — Data Quality Engineer	Roles 1, 3
Prometheus configuration	Role 4 — Observability Engineer	Roles 3, 6
Grafana dashboard	Role 4 — Observability Engineer	Roles 3, 5
Alert examples	Role 4 — Observability Engineer	Roles 3, 5
Failure-recovery demonstration	Role 3 — Orchestration Engineer	Roles 4, 5
Documentation	Role 6 — DevOps Engineer	All roles
Docker Compose — optional	Role 6 — DevOps Engineer	Roles 1, 3, 4
GitHub Actions — optional	Role 6 — DevOps Engineer	Role 5

Risk Assessment & Mitigation Plan

1. Executive Summary

This document provides a comprehensive Risk Assessment and Mitigation Plan for the **Data Quality & Pipeline Observability Platform**. The objective of this framework is to proactively identify potential operational, data integrity, and system-level failures, and to define clear mitigation strategies and contingency mechanisms. By addressing these risks, we ensure automated data quality enforcement, reliable ETL processing, and continuous observability.

2. Risk Matrix & Detailed Assessment

Risk ID	Category	Risk Description	Likelihood	Impact	Severity	Mitigation Strategy	Mitigation Plan / Contingency
R-01	Data Quality	Data Drift & Schema Mismatch Upstream data changes schema or types unexpectedly.	Medium	**High**	**High**	Implement automated pre-ingestion validation (Schema/Type, Nulls, Ranges).	Quarantine invalid records into a Dead-Letter Queue / Error Table and notify engineers via alerts.
R-02	Pipeline Reliability	Pipeline Task Failures Airflow DAG execution fails due to network timeouts or memory limits.	**High**	Medium	**High**	Define retry policies with backoff intervals and configure resource limits in Docker.	Route failed task outputs to dead-letter storage, trigger alerts, and run automatic backfill/retries.
R-03	Observability	Monitoring Infrastructure Outage Prometheus or Grafana instance becomes unavailable, leading to metrics loss.	Low	**High**	Medium	Deploy monitoring stack in containerized isolation with volume persistence.	Fallback to file-based pipeline execution logs for post-incident audit and debugging.
R-04	Performance	Validation Bottlenecks & High Latency Data quality checks significantly delay large batch ETL runs.	Medium	Medium	Medium	Optimize validation logic, use database indexing, and process checks in parallel/batches.	Scale worker nodes horizontally using Docker Compose / Celery executors.
R-05	Security & Compliance	Sensitive Data Leakage in Logs Rejected records or stack traces expose sensitive/unmasked data in logs.	Low	**Critical**	**High**	Mask or sanitize error payloads before writing to dead-letter tables or logs.	Enforce Role-Based Access Control (RBAC) on Grafana dashboards and lock down .env

3. Sub-System Risk Mitigation Breakdown (In-Depth Analysis)

A. Data Quality Engine & Validation Layer

The Data Quality Engine forms the primary line of defense against data corruption. Upstream data pipelines frequently experience unexpected schema changes, missing null constraints, out-of-range values, or duplicate records. Without robust pre-ingestion gates, these anomalies contaminate data warehouses, breaking downstream BI dashboards, machine learning models, and analytics workflows.

Detailed Risk Scenario: Bad or malformed data (e.g., unexpected data types, primary key duplicates, broken foreign key constraints, or nulls in non-nullable columns) leaking directly into production analytical stores.

Comprehensive Mitigation Protocols:

Non-Blocking Data Routing Gates: Implement row-level isolation mechanics. Valid records meeting all schema and quality constraints are immediately routed downstream to the target data warehouse, ensuring zero processing delays for good data. Non-compliant records are diverted into an isolated dead-letter quarantine store (e.g., rejected_records table or bucket) along with explicit error codes and metadata.

Automated Pre-Ingestion Rules Engine: Enforce strict schema validation rules (data type matching, mandatory null checks, numerical boundary limits, string pattern matching, and referential integrity) using framework-native validations (e.g., Great Expectations, PySpark assertions, or SQL constraints) prior to writing into analytical layers.

Threshold-Based Alerting & Circuit Breakers: Configure real-time monitoring on quarantine volumes. If the rejection rate exceeds defined error budgets (e.g., > 5% of a batch batch size), automated circuit breakers trigger to halt pipeline execution, preventing systemic failure cascades and alerting data engineers immediately.

B. Pipeline Reliability & Apache Airflow Orchestration

Data orchestration frameworks rely on predictable execution and fault tolerance. In large-scale environment executions, temporary network instability, database connection timeouts, memory spikes (OOM errors), or third-party API rate limits can cause unexpected task failures. Unhandled exceptions stall downstream DAG dependencies, leading to stale data and missed SLAs.

Detailed Risk Scenario: Unhandled operator exceptions causing critical DAG execution paths to fail or freeze, blocking downstream reporting pipelines and creating backlogs.

Comprehensive Mitigation Protocols:

Resilient Retry Policies with Exponential Backoff: Standardize default task parameters across all custom Airflow operators and DAG definitions. Configure task retries (e.g., retries=3) combined with exponential backoff intervals and jitter to avoid overwhelming recovering external services or transient network hiccups.

Standardized Exception Handling & Callback Hooks: Wrap task execution code within robust error handles. Utilize Airflow's on_failure_callback and on_retry_callback hooks to log structured execution state, send real-time notifications to Slack/PagerDuty, and push failure metrics directly to Prometheus for tracking.

Resource Management & Isolation: Define strict CPU and Memory requests and limits for Docker containers hosting Airflow Celery Workers. Enforce task-level memory boundaries to prevent single worker node crashes from affecting concurrent task executions.

C. Observability, Metrics & Alert Management

Observability provides end-to-end operational visibility into pipeline health, processing durations, and record volumes. However, misconfigured alerting systems frequently generate high volumes of false-positive warnings, leading to alert fatigue among engineering teams. Conversely, unmonitored infrastructure outages cause undetected data gaps.

Detailed Risk Scenario: Alert fatigue resulting from excessive minor notifications, or monitoring system downtime masking critical pipeline failures.

Comprehensive Mitigation Protocols:

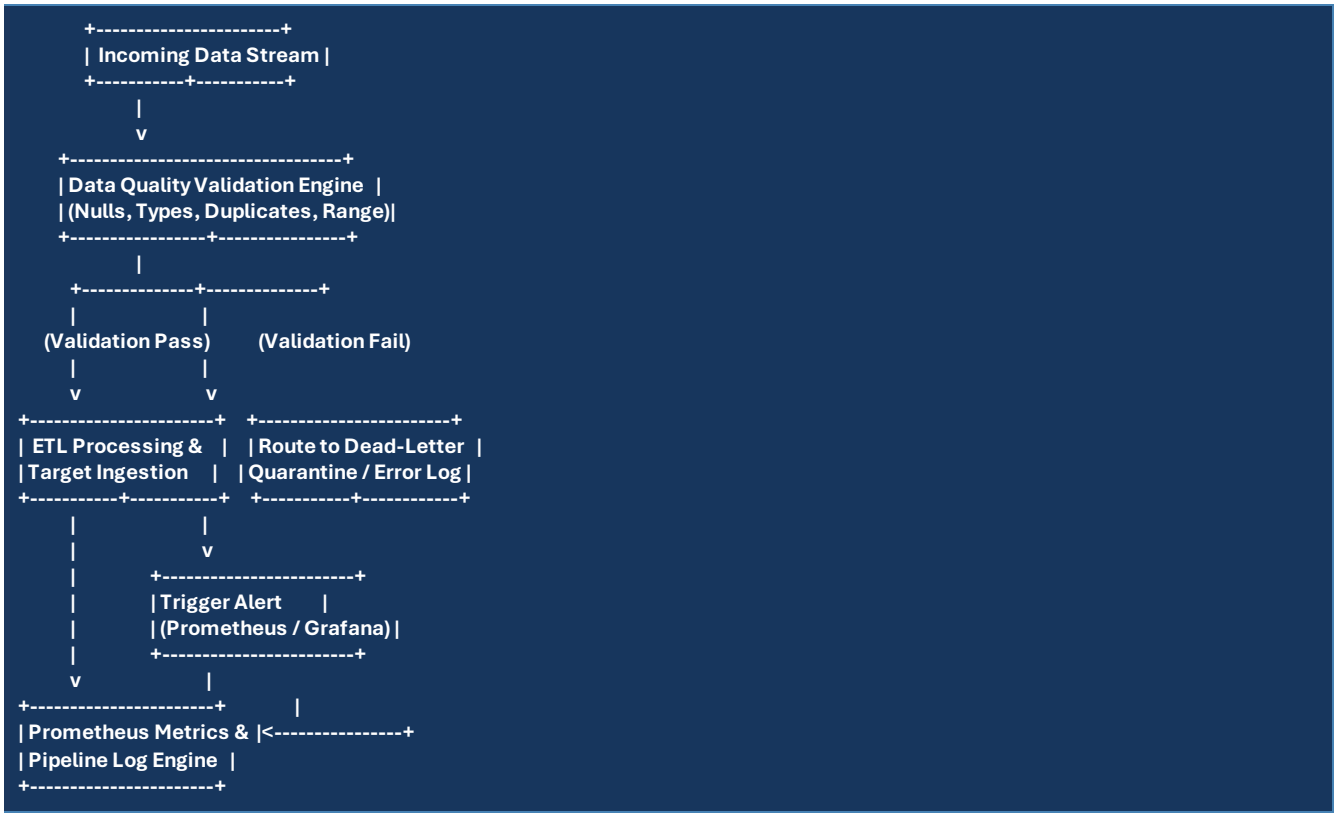
Tiered Alert Severity Taxonomy: Establish strict incident classification to ensure appropriate engineering response:
- Critical (P1 - Immediate Action Required): Total pipeline failure, database target unreachable, or schema breaking changes halting execution.
- Warning (P2 - Attention Needed): Non-fatal quality anomaly threshold reached (>5% rejection rate), or pipeline duration exceeding standard baseline by 30%.
- Info (P3 - Audit Log Only): Standard task completion, minor retries, or batch size statistics.

Dynamic Alert Baseline Calibration: Fine-tune Grafana and Alertmanager alert rules based on historical rolling averages of execution duration and processing volume, reducing noise during expected traffic spikes.

Decentralized Fallback Logging: In the event of Prometheus or Grafana monitoring stack outages, maintain local structured JSON log files on persistent storage to allow continuous auditing and post-incident analysis.

4. Disaster Recovery & Fallback Workflow (Detailed Mechanism)

The platform implements an automated end-to-end fail-safe operational flow. As data enters the system, it passes through an isolated pre-validation layer. Records satisfying all rules proceed to target ingestion, while invalid data is automatically captured in dead-letter storage with zero loss to pipeline momentum. The diagram and accompanying workflow steps below articulate this architecture.



Step-by-Step Disaster Recovery Procedure:

1. Ingestion & Quality Evaluation: Incoming batches are received and evaluated against schema, duplicate key checks, null contraints, and range checks before any database write occurs.

2. Dual-Stream Routing: Passing rows move forward into the core ETL processing phase. Failed rows are isolated into dead-letter tables with audit timestamps and exact reason tags.

3. Alert Triggering & Incident Handling: If dead-letter volume surpasses warning thresholds, Prometheus triggers Grafana/PagerDuty alerts. On-call engineers receive actionable context with direct links to quarantined record samples.

4. Backfill & Re-Ingestion: Once schema adjustments or data cleanups occur, engineers execute automated Airflow backfill DAGs to re-process quarantined records from dead-letter storage back into the core stream.

5. Tools, Data Sources & Risk Alignment

To ensure end-to-end risk mitigation, all core deliverables, tools, and data sources in the platform architecture are mapped directly to their operational purpose and the specific risks they address:

Tool / Data Source	Purpose	Targeted Risk Mitigation
CI test reports	Record automated test results and identify failures.	**R-01 (Data Drift & Schema Mismatch)** **R-02 (Pipeline Task Failures)**
PostgreSQL	Store pipeline run history, execution duration, statuses, and relevant batch results.	**R-02 (Pipeline Reliability & Failure Audit)** **R-04 (Performance Monitoring)**
Python structured logs	Record execution events, exceptions, and troubleshooting information.	**R-02 (Task Exception Analysis)** **R-05 (Sanitized Error Payload Auditing)**
Prometheus	Collect and expose operational metrics for monitoring.	**R-03 (Metrics Loss & Infrastructure Health)** **R-04 (High Latency Bottlenecks)**
Grafana	Visualize available metrics and display configured alerts.	**R-02 (Real-time Pipeline Alerting)** **R-03 (Observability Visualization)**
Requirement traceability matrix	Connect functional requirements to implementation modules and tests.	**R-01 (Schema & Logic Verification)** **R-05 (Security & Compliance Mapping)**
Operational runbook	Document setup, execution, troubleshooting, and recovery procedures.	**R-02 (Task Failure Recovery & Backfill)** **R-03 (Infrastructure Outage Fallback)**

6. Summary of Deliverables Alignment

This risk assessment framework directly supports and reinforces all business and technical deliverables required for the Data Quality & Pipeline Observability Platform:

Automated Data Quality & Schema Enforcement: Ensures zero corrupt or schema-mismatched records reach analytical target databases through pre-ingestion gates, dead-letter quarantining, and threshold alerts.

Pipeline Reliability & High Availability: Provides operational resiliency using exponential backoff retries, resource boundary limits, and standardized exception handling across Airflow DAGs.

Comprehensive Observability & Alerting Framework: Delivers end-to-end visibility into pipeline runtimes, execution pass/fail statuses, record throughput, and actionable tiered alerting with fallback logging.

Data Security & Audit Compliance: Protects sensitive log data through payload masking, strict environment secrets management, and role-based access control (RBAC).

KPIs (KEY PERFORMANCE INDICATORS)

KPIs and Measurement Plan

Author: Sara Ahmed Abdelhamid

1. Purpose

The purpose of this document is to define the Key Performance Indicators (KPIs) used to evaluate the quality, reliability, performance, and observability of the Data Quality & Pipeline Observability Platform.

These KPIs provide measurable acceptance criteria to verify that the pipeline detects data-quality issues, handles failures correctly, prevents duplicate records, records execution results, and exposes useful monitoring information.

2. KPI Summary Table

KPI ID	KPI Name	Definition / Measurement Method	Target	Measurement Frequency
KPI-01	Data Quality Check Coverage	Number of implemented and tested data-quality rule families divided by the six required rule families.	6 out of 6 rule families implemented and tested.	At pull request approval and release review.
KPI-02	Automated Test Pass Rate	Number of passed required automated test cases divided by the total number of executed required test cases.	100% of required tests pass, with no unexplained failures or skipped required tests.	During every CI run or test execution.
KPI-03	Failure Detection and Recovery	Verify that a batch containing invalid data produces a FAILED run and quarantines invalid records, and that a corrected batch can complete successfully.	All required failure and recovery scenarios pass.	During integration testing and final demonstration.
KPI-04	Data Rejection Rate	Percentage of input rows rejected by data-quality validation, calculated using the total number of input rows inspected.	No more than 10% for a predefined normal test batch.	After every batch execution.
KPI-05	Data Load Idempotency	Number of duplicate destination primary-key records created after rerunning the same batch.	Zero duplicate destination primary-key records after a rerun.	During regression and rerun tests.
KPI-06	Batch Execution Duration	Elapsed time between the start and completion of a pipeline run, recorded in pipeline_runs.duration_seconds and exposed through the relevant Prometheus metric.	Establish a baseline and aim for 60 seconds or less for the defined demonstration dataset and test environment.	After every batch execution.
KPI-07	Pipeline Run Outcome Visibility	Verify that the latest run status is visible and that run outcomes are stored in the database for historical review.	Every completed run has a recorded SUCCESS or FAILED status, with the latest outcome available through the designated monitoring interface.	After every batch execution.
KPI-08	Alert Scenario Coverage	Number of required alert scenarios configured and successfully tested: failed pipeline, stale pipeline, pipeline that has never reported a run, and high data-rejection rate.	All four alert scenarios are configured and their firing conditions are tested.	During monitoring tests and before delivery.
KPI-09	Functional Requirement Traceability	Percentage of applicable functional requirements mapped to implementation modules and corresponding tests.	At least 90% of applicable requirements are mapped, with all critical requirements covered.	During project reviews and before release.
KPI-10	Operational Runbook Readiness	Verify that the documentation includes environment setup, pipeline execution, troubleshooting, and failure-recovery instructions.	All required runbook sections are complete and successfully followed by a reviewer.	Before final submission.

3. KPI Measurement Details

KPI-01: Data Quality Check Coverage

The platform must implement and test the following six data-quality rule families:

1. **Null Value Checks:** Detect missing values in required fields.
2. **Duplicate Checks:** Identify duplicate records according to the defined business or primary-key rules.
3. **Schema and Data Type Validation:** Verify that the input data follows the expected schema and data types.
4. **Row Count Checks:** Validate that the batch contains an acceptable number of records.
5. **Referential Integrity Checks:** Verify that references to related records are valid.
6. **Range Checks:** Detect values that fall outside predefined acceptable limits.

Calculation:

Data Quality Check Coverage = (Implemented and Tested Rule Families / 6) × 100

Target: 100% coverage, equivalent to six out of six rule families.

Evidence should include automated tests or documented demonstrations for each rule family.

KPI-02: Automated Test Pass Rate

This KPI measures whether the required automated tests pass successfully.

Calculation:

Test Pass Rate = (Number of Passed Required Tests / Total Number of Executed Required Tests) × 100

Target: 100%.

The required test suite should be defined in advance. Failed tests must be investigated, and required tests must not be excluded or skipped merely to achieve the target.

Evidence can be obtained from test reports and continuous integration logs.

KPI-03: Failure Detection and Recovery

This KPI verifies that the platform handles invalid data and recovers after the underlying problem is corrected.

The demonstration should follow these steps:

1. Submit a batch containing invalid records.
2. Verify that the pipeline records a FAILED run when the configured failure condition is met.
3. Verify that invalid records are stored in the designated quarantine or error store.
4. Correct the input data or resolve the underlying issue.
5. Rerun the pipeline.
6. Verify that the corrected batch completes successfully and that the final run status is SUCCESS.

Target: All required failure-detection and recovery scenarios pass.

Evidence should include run-status records, error or quarantine records, and execution logs.

KPI-04: Data Rejection Rate

This KPI measures the percentage of input records rejected during data-quality validation.

Calculation:

Data Rejection Rate = (Rejected Input Rows / Total Input Rows Inspected) × 100

Target: 10% or less for a predefined normal test batch.

The denominator must represent all input rows inspected, including both accepted and rejected rows.

The normal test batch must be defined before measurement. Intentionally corrupted test batches should be evaluated separately because they are designed to trigger validation failures.

If a batch contains zero input rows, the rejection rate should be recorded as not applicable, and the appropriate empty-batch or row-count validation should be evaluated separately.

Evidence should include batch-level input and rejection counts.

KPI-05: Data Load Idempotency

This KPI checks whether rerunning the same batch creates duplicate records in the destination database.

Measurement Procedure:

1. Run the batch and record the destination records.
2. Run the same batch again.
3. Check the destination table for duplicate primary keys.
4. Compare the results to verify that rerunning the batch does not create unintended duplicates.

Target: Zero duplicate destination primary-key records caused by the rerun.

Evidence should include database queries and test results.

KPI-06: Batch Execution Duration

This KPI measures the time required for a pipeline run.

Calculation:

Batch Duration = Run Completion Time − Run Start Time

The duration should be measured using a monotonic clock in the application and recorded in pipeline_runs.duration_seconds. The latest-run duration may also be exposed as a Prometheus gauge.

Target: Establish a baseline and aim for a duration of 60 seconds or less for the demonstration dataset.

The 60-second target is provisional. It must be evaluated using a predefined dataset size and a documented test environment. It should not be treated as a production performance guarantee.

Evidence should include execution records and the corresponding monitoring metric.

KPI-07: Pipeline Run Outcome Visibility

This KPI verifies that pipeline execution outcomes can be inspected after a run finishes.

The platform should record the status of each run in the database and expose the latest run outcome through the designated monitoring interface, such as a Grafana dashboard where configured.

Target: Every completed run has a recorded SUCCESS or FAILED status, and the latest outcome is available for review.

Evidence should include database records and a screenshot or demonstration of the monitoring interface.

KPI-08: Alert Scenario Coverage

This KPI evaluates whether the monitoring system can detect and report the required operational conditions.

The four required alert scenarios are:

1. **Failed Pipeline:** An alert is triggered when a pipeline run fails.
2. **Stale Pipeline:** An alert is triggered when the pipeline has not completed a successful run within the configured time threshold.
3. **Never-Reported Pipeline:** An alert is triggered when expected pipeline metrics have never been reported.
4. **High Data-Rejection Rate:** An alert is triggered when the rejection rate exceeds the configured threshold.

For a daily pipeline, a stale-run threshold of 26 hours may be used as an initial configuration, allowing a small tolerance beyond the expected 24-hour schedule. This threshold should be adjusted to match the final scheduling requirements.

Target: All four alert scenarios are configured, and each firing condition is tested.

Evidence should include alert-rule configuration and test results showing that each condition produces the expected alert.

KPI-09: Functional Requirement Traceability

This KPI measures how effectively the project's functional requirements are connected to the implementation and testing activities.

Calculation:

Requirement Traceability = (Applicable Requirements Mapped to Modules and Tests / Total Applicable Requirements) × 100

Target: At least 90% of applicable requirements are mapped, and all critical requirements are covered.

The traceability record should identify the requirement, implementation module, corresponding test, and test result.

Evidence should include a requirement traceability matrix.

KPI-10: Operational Runbook Readiness

This KPI verifies that another team member can set up, run, troubleshoot, and recover the pipeline by following the documentation.

The runbook must include:

1. Prerequisites and environment setup.
2. Dependency installation and configuration.
3. Database and monitoring setup, where applicable.
4. Instructions for manually triggering a pipeline run.
5. Instructions for inspecting logs and run statuses.
6. Common failure scenarios and troubleshooting steps.
7. Procedures for correcting input data and rerunning a failed batch.
8. Instructions for verifying successful recovery.

Target: All required sections are complete, and a reviewer can follow the documented procedures successfully.

Evidence should include the completed runbook and a documented review or demonstration.

4. Measurement Tools and Data Sources

The following tools and data sources will support KPI measurement:

Tool / Data Source	Purpose
Python automated tests	Verify validation rules, transformations, failure handling, and recovery behavior.
CI test reports	Record automated test results and identify failures.
PostgreSQL	Store pipeline run history, execution duration, statuses, and relevant batch results.
Python structured logs	Record execution events, exceptions, and troubleshooting information.
Prometheus	Collect and expose operational metrics for monitoring.
Grafana	Visualize available metrics and display configured alerts.
Requirement traceability matrix	Connect functional requirements to implementation modules and tests.
Operational runbook	Document setup, execution, troubleshooting, and recovery procedures.

Historical run statistics should be calculated from the database records when historical information is required. A Prometheus gauge representing the latest batch is useful for current-state monitoring but does not, by itself, provide complete historical run counts or success rates.

5. KPI Evaluation and Evidence Collection

KPIs should be evaluated at the appropriate stage of the project:

- **During development:** Implement validation rules and create automated tests.
- **During integration testing:** Verify database loading, failure handling, quarantine behavior, and recovery.
- **During monitoring setup:** Configure Prometheus metrics and Grafana alert rules, then test their firing conditions.
- **Before release:** Review requirement traceability, runbook completeness, and all required acceptance evidence.
- **During the final demonstration:** Present the test results, database records, execution logs, monitoring metrics, and relevant dashboard or alert evidence.

Each KPI should be marked as **Not Tested**, **Passed**, or **Failed** based on actual evidence. A proposed target must not be reported as achieved until the corresponding measurement or test has been completed.

6. Release Acceptance Criteria

The platform will be considered ready for submission when the following acceptance criteria have been demonstrated:

1. All six required data-quality rule families are implemented and tested.
2. All required automated tests pass.
3. Failure detection, quarantine, and recovery scenarios work as expected.
4. Rerunning the same batch does not create unintended duplicate destination records.
5. Every completed run has a recorded outcome, and the latest outcome is visible through the designated interface.
6. All four required alert scenarios are configured and tested.
7. All critical functional requirements are linked to implementation and test evidence.
8. The operational runbook is complete and has been successfully reviewed.

The rejection-rate target must be evaluated using a predefined normal test batch. Batch-duration performance must be evaluated using a documented dataset size and environment. These measurements should be reported separately from the other acceptance criteria.

7. Conclusion

The KPI and Measurement Plan establishes a practical framework for evaluating the reliability, data quality, performance, and observability of the Data Quality & Pipeline Observability Platform.

By using measurable targets and verifiable evidence, the project team can determine whether the pipeline detects data-quality issues, handles failures, supports safe reruns, records execution history, and provides useful monitoring and alerting capabilities.

The KPIs also help the team identify implementation gaps before submission and provide a clear basis for demonstrating that the platform meets its intended objectives.

System Architecture, Literature Review & Software Requirements Specification (SRS)

Author: Elhussieny Sabry Elshahat

1. Executive Summary & Problem Formulation

Modern data-driven enterprises increasingly rely on complex, multi-stage automated ELT/ETL pipelines feeding operational dashboards, machine learning models, and executive analytics. However, a pervasive systemic failure mode persists across organizations: silent data corruption, schema drift, referential integrity mismatches, and pipeline execution failures are routinely detected reactively—often only after business stakeholders report broken or contradictory dashboards. This reactive paradigm erodes organizational data trust, incurs severe operational remediation costs, and impairs strategic decisions.

To resolve this industry-wide challenge, the Data Quality & Pipeline Observability Platform establishes an end-to-end, production-grade data engineering framework. It combines preventative inline multi-family data quality validation, automated quarantine and dead-letter queue (DLQ) routing, resilient orchestration via Apache Airflow with intelligent retry backoff, continuous telemetry exporting through Prometheus, and real-time visualization alongside alerting via Grafana.

2. Comprehensive Literature Review & Comparative Analysis

A rigorous state-of-the-art review was conducted across open-source architectures, production reference implementations, and real-world repositories to analyze existing paradigms in data quality validation and pipeline telemetry. Three primary projects were analyzed in detail to benchmark architectural tradeoffs and identify technical gaps.

2.1 Project Reference A: RetailFlow Pipeline (elhussienysabry/retailflow-pipeline)

• Repository: https://github.com/elhussienysabry/retailflow-pipeline
• Core Technologies: Python, PostgreSQL, dbt (Data Build Tool), Great Expectations, Pandera, Apache Airflow, Docker, GitHub Actions.
• Architectural Highlights: RetailFlow is an end-to-end retail data platform implementing Slowly Changing Dimensions (SCD Type 2) and historical snapshots via dbt. Data validation is implemented using two leading declarative libraries: Pandera for schema enforcement and Great Expectations for data contract validation.
• Strengths: Exceptional schema enforcement and data modeling rigor; robust multi-stage contract testing; clean CI/CD automation.
• Architectural Limitations & Gaps: Despite strong static and batch assertion capabilities, RetailFlow lacks an active, live telemetry observability stack (no Prometheus metrics exporter or real-time scraping). Validation failures halt the transformation or require manual inspection rather than automatically routing corrupted records to a dedicated Dead-Letter Queue (DLQ) quarantine store while allowing healthy records to proceed safely. Furthermore, it does not provide custom alerting for stale or abnormal execution patterns.

2.2 Project Reference B: Airflow Data Quality Demo (astronomer/airflow-data-quality-demo)

• Repository: https://github.com/astronomer/airflow-data-quality-demo
• Core Technologies: Apache Airflow, Great Expectations Provider, SQL Check Operators, Snowflake / BigQuery.
• Architectural Highlights: Astronomer's reference repository illustrates modern DAG patterns using Airflow's native SQL operators (ValueCheckOperator, IntervalCheckOperator) and Great Expectations task operators to gate downstream warehouse updates.
• Strengths: Industry-standard task dependency sequencing; effective fail-fast DAG execution stopping corrupted batches before target ingestion.
• Architectural Limitations & Gaps: The project implements an 'all-or-nothing' paradigm: any single record anomaly causes the entire DAG task to fail, creating massive pipeline backpressure in high-volume production environments. It lacks row-level quarantine mechanisms, does not export operational metrics to time-series monitoring engines (Prometheus), and lacks automated failure-recovery testing workflows.

2.3 Project Reference C: Airflow Prometheus & Grafana Monitoring (alfredomarinus/airflow-prometheus-grafana)

• Repository: https://github.com/alfredomarinus/airflow-prometheus-grafana
• Core Technologies: Apache Airflow, Prometheus StatsD Exporter, Grafana, Docker Compose.
• Architectural Highlights: Focuses on infrastructure-level observability by translating Airflow internal daemon StatsD

emissions into Prometheus scrapeable metrics, paired with pre-configured Grafana operational dashboards.
• Strengths: Outstanding visualization of infrastructure resource metrics, executor task slots, DAG execution heartbeats, and cluster health.
• Architectural Limitations & Gaps: The telemetry is strictly operational and infrastructure-centric (CPU, memory, worker saturation, Airflow task states). It possesses zero semantic visibility into data payload health (no record throughput counters, rejection percentage rates, validation failure categories, or field-level corruption distributions). Data quality engineering is entirely absent from this implementation.

2.4 Systematic Comparison Matrix

Capability / Architectural Dimension	RetailFlow Pipeline	Astronomer DQ Demo	Airflow-Prom-Grafana	Our Proposed Platform
Orchestration Engine	Apache Airflow	Apache Airflow	Apache Airflow	**Apache Airflow (Modular DAGs)**
Data Quality Scope	Pandera / Great Expectations	Great Expectations / SQL Checks	None (No DQ logic)	**Full 6-Family Modular DQ Suite**
Error Handling Strategy	Pipeline Halt / Log inspection	Fail-fast Task Abortion	Task Retry on Pod crash	**Dead-Letter Quarantine (DLQ) + Error Store**
Idempotency Mechanism	dbt Snapshots / Merge	Warehouse Staging Merge	N/A	**Atomic Upsert / Primary Key Deduplication**
Telemetry Engine	None	None	Prometheus (StatsD Exporter)	**Prometheus (Custom Payload Exporter)**
Observability Granularity	Batch Execution logs	Task execution states	Infra & Task Heartbeats	**Semantic Data Metrics + Duration + Rejection Rates**
Operational Dashboard	None	Airflow Web UI	Grafana (Airflow Cluster)	**Grafana (Executive DQ & Pipeline Health)**
Automated Alerting	None	Airflow Email On Failure	Prometheus Alertmanager (Infra)	**4 Multi-scenario DQ & Health Alerts**
Demonstrated Self-Healing	Manual intervention	Manual intervention	Pod auto-restart	**Reproducible Failure & Recovery Demonstration**
CI/CD & Containerization	Docker & GitHub Actions	Dockerfile only	Docker Compose	**Docker Compose Stack + GitHub Actions CI**

2.5 Architectural Synthesis & Technical Gap Formulation

The literature review reveals an evident architectural bifurcation in current industry solutions:
1. Data Quality tools (Pandera, Great Expectations) focus purely on statistical rules and assertions, remaining blind to operational time-series telemetry and dashboarding.

2. Infrastructure Monitoring tools (Prometheus, StatsD, Grafana) observe server uptime and worker CPU, remaining blind to payload corruption, rejection ratios, and bad business logic.

Our platform bridges this exact chasm. By implementing inline 6-family validation, dead-letter quarantine routing, idempotent upserts, and programmatic Prometheus metric emission within an Airflow DAG, the system provides unified 'Payload-to-Platform' observability.

3. Requirements Gathering & Technical Specifications

Requirements were elicited and structured across stakeholders, functional modules, and operational quality constraints. The engineering responsibilities map directly to the 6 project engineering roles and the 10 measurable Key Performance Indicators (KPIs).

3.1 Stakeholder Profiles & Core Value Propositions

Stakeholder Group	Primary Responsibilities	Core System Requirements & Value Elicited
Data Engineers & Pipeline Developers	Pipeline creation, maintenance, ETL execution	Needs idempotent loading, automated quarantine for corrupted records without aborting the batch, and exact processing counters.
Business & Analytics Stakeholders	Dashboard consumption, business decisions, reporting	Demands zero corrupted or duplicate records in gold tables, and guaranteed SLA with automated anomaly detection before dashboards render.
DevOps & SRE Engineers	Platform uptime, infrastructure monitoring, deployment	Requires standardized Docker Compose execution, continuous Prometheus scraping, live Grafana dashboards, and automated CI/CD validation.

3.2 Detailed Functional Requirements (FR) by Role Scope

3.2.1 Data Pipeline Engineering (Role 1 - [DE])

• **FR-DE-01 (Data Modeling & Contracts):** Define input dataset specifications and production target schema in PostgreSQL, including primary keys, foreign keys, constraints, and the 'pipeline_runs' execution metadata table (KPI-09).

• **FR-DE-02 (Robust Source Extraction):** Implement modular extraction logic capable of parsing source files/tables while gracefully handling missing inputs or zero-row empty batches without unhandled crashes.

• **FR-DE-03 (Idempotent Target Loading):** Implement atomic Upsert / Merge logic ensuring that rerunning identical batches results in zero duplicate primary keys in target tables (KPI-05).

• **FR-DE-04 (Processing Instrumentation & Hooks):** Expose granular instrumentation hooks calculating exact counts for records extracted, inspected, successfully loaded, and quarantined for downstream orchestration and telemetry (KPI-04).

• **FR-DE-05 (Synthetic Benchmark Datasets):** Generate standard valid test batches (achieving <= 10% rejection rate for KPI-04) and intentionally corrupted batches containing nulls, duplicates, invalid schemas, range violations, and foreign-key breaks.

3.2.2 Data Quality Validation Engine (Role 2 - [DQ])

• **FR-DQ-01 (Null-Value Validation):** Inspect and enforce presence constraints on mandatory non-nullable business fields (KPI-01).

• **FR-DQ-02 (Duplicate Detection):** Detect duplicated primary records and candidate business keys before target table ingestion (KPI-01).

• **FR-DQ-03 (Schema & Type Validation):** Enforce strict schema compliance, verifying column presence, ordering, and data type compatibility (KPI-01).

• **FR-DQ-04 (Row-Count & Range Validation):** Validate batch volume boundaries (minimum/maximum row thresholds) and numerical/date range boundaries (e.g., non-negative prices, valid transaction dates) (KPI-01).

• **FR-DQ-05 (Referential Integrity Checks):** Enforce foreign-key relationships against reference/dimension tables, rejecting orphaned records (KPI-01).

• **FR-DQ-06 (Dead-Letter Quarantine Storage):** Automatically isolate invalid records into a dedicated error table/file with run IDs, timestamps, and explicit failure reasons without terminating the healthy batch (KPI-03, KPI-04).

• **FR-DQ-07 (ETL Pipeline Integration):** Embed all six validation categories inline into the ETL execution flow producing deterministic pass/fail outcomes (KPI-01).

3.2.3 Orchestration, Reliability & Failure Recovery (Role 3 - [OR])

• **FR-OR-01 (Airflow DAG Sequencing):** Construct an Apache Airflow DAG enforcing strict task dependencies: Extract -> Validate -> Quarantine/Transform -> Load -> Emit Metrics.

• **FR-OR-02 (Exception Handling & Backoff Retries):** Configure exponential backoff retry policies for transient network/DB errors, while permanently failing on fatal schema/contract breaches without silent success (KPI-03, KPI-07).

• **FR-OR-03 (Execution Metadata Logging):** Persist comprehensive execution metadata in PostgreSQL: run_id, DAG status (SUCCESS/FAILED), start/end timestamps, duration_seconds, row counters, and error traces (KPI-06, KPI-07).

• **FR-OR-04 (Deterministic Failure Recovery):** Provide an automated recovery workflow demonstrating that a corrupted batch is halted and quarantined, and upon data correction, rerunning the batch transitions the state to SUCCESS and updates metrics (KPI-03).

3.2.4 Observability, Telemetry & Alerting (Role 4 - [OB])

• **FR-OB-01 (Prometheus Metrics Exporter):** Expose time-series metrics including pipeline duration (seconds), cumulative success/failure run counters, total records processed vs. rejected, and DQ failures grouped by rule category (KPI-01, KPI-04, KPI-06, KPI-07).

• **FR-OB-02 (Prometheus Scrape Configuration):** Maintain prometheus.yml configuring automated scraping intervals against the pipeline metrics exporter.

• **FR-OB-03 (Operational Grafana Dashboard):** Design and provision an executive Grafana dashboard with live visual panels for pipeline duration trends, run status gauges, processed vs. rejected record ratios, and DQ violation heatmaps (KPI-07).

• **FR-OB-04 (Multi-Scenario Alert Rules):** Configure and demonstrate four operational alert scenarios: (1) Failed Pipeline Run, (2) Stale Pipeline (>26h without success), (3) Never-Reported Pipeline (missing heartbeat), and (4) High Data-Rejection Rate (>10%) (KPI-08).

3.2.5 Automated Testing & Quality Assurance (Role 5 - [QA])

• **FR-QA-01 (Pytest DQ Unit Suite):** Implement comprehensive unit tests for all six validation categories, ensuring >= 1 passing test and >= 1 failing test per category (KPI-01, KPI-02).

• **FR-QA-02 (Integration & Idempotency Testing):** Verify end-to-end batch processing, asserting zero duplicate primary keys on reruns and complete isolation of quarantined rows (KPI-02, KPI-05).

• **FR-QA-03 (Telemetry Reconciliation & Alert Testing):** Assert 100% numerical reconciliation between PostgreSQL logs and Prometheus metrics, and trigger all 4 alert rules to verify firing behavior (KPI-02, KPI-07, KPI-08).

• **FR-QA-04 (Traceability Matrix & Test Report):** Produce formal verification reports linking >= 90% of all requirements and 100% of critical requirements to passing automated test cases (KPI-02, KPI-09).

3.2.6 DevOps, Containerization & CI/CD (Role 6 - [DO])

• **FR-DO-01 (Repository Architecture):** Establish standardized multi-directory repository structure (etl/, dags/, validation/, monitoring/, tests/, config/, docs/).

• **FR-DO-02 (Secret Management & Configuration):** Externalize all database passwords, API keys, and connection strings into .env templates without hardcoded secrets.

• **FR-DO-03 (Docker Compose Multi-Container Stack):** Package PostgreSQL, Airflow (Webserver + Scheduler), Prometheus, Grafana, and Exporters with integrated Docker health checks (DO-06).

• **FR-DO-04 (GitHub Actions CI Automation):** Configure automated GitHub Actions workflow executing code linting, formatting, and pytest automated test suites on every pull request (DO-07).

• **FR-DO-05 (Documentation & Operational Runbook):** Provide reproducible clean-slate setup instructions, architecture schematics, and an operational troubleshooting runbook (KPI-10).

3.3 Non-Functional Requirements (NFR) & Quality Attributes

Attribute	Specification & Quality Objective	Measurement Metric / KPI Target
Performance & Latency	End-to-end execution of the demonstration batch (extraction, validation, loading, metric emission) must complete within a strict operational window.	**Total execution duration <= 60 seconds (KPI-06).**
Data Integrity & Reliability	Zero corrupted records may leak into the production target schema. Corrupted batches must fail loudly while healthy batches maintain low rejection.	**Rejection rate <= 10% on clean batches; 0 duplicate PKs (KPI-04, KPI-05).**
Fault Tolerance & Self-Healing	Transient database connection or lock timeouts must trigger retry backoffs. Permanent validation errors must be isolated to DLQ.	**Deterministic recovery with verifiable state flip from FAILED to SUCCESS (KPI-03).**
Security & Compliance	Zero plain-text credentials or API secrets committed to source control. Role-Based Access Control on Grafana and database layers.	**100% externalized environment variables via .env templates (KPI-09, DO-03).**
Testability & Maintainability	Modular code architecture with automated testing covering all data quality validation rules and pipeline failure modes.	**100% pass rate across automated pytest suites; >=90% requirement traceability (KPI-02, KPI-09).**
Portability & Deployability	The complete software system must be runnable from a completely clean machine using standard virtualization tooling without host dependency pollution.	**One-click 'docker compose up' spin-up with passing health checks (DO-06, KPI-10).**

4. Requirements-to-KPI Traceability Matrix (RTM)

In compliance with Phase 6 deliverables (QA-07 and DO-05), the matrix below maps each of the 10 project Key Performance Indicators (KPIs) directly to its corresponding functional tasks, requirements, lead engineering role, and verifiable acceptance criteria.

KPI ID	Measurable Target Objective	Lead Role	Mapped Tasks & Requirements	Verification Criterion
KPI-01	6/6 Data Quality Rule Families	**Role 2 [DQ]**	DQ-01 to DQ-05, DQ-07 (FR-DQ-01)	All 6 checks pass on clean data and accurately detect injected flaws.
KPI-02	100% Automated Test Pass Rate	**Role 5 [QA]**	QA-01 to QA-06 (FR-QA-01, FR-QA-02)	Pytest suite executes with zero failures in local and CI runs.
KPI-03	Failure Recovery Demonstration	**Role 3 [OR]**	OR-04, OR-07, DQ-06 (FR-OR-04)	Bad batch marked FAILED; corrected batch reruns to SUCCESS.
KPI-04	Normal Batch Rejection Rate <= 10%	**Role 2 [DQ]**	DE-05, DE-06, DQ-06 (FR-DE-04)	Normal batch quaratines <= 10% invalid rows without pipeline crash.
KPI-05	Zero Duplicate Primary Keys	**Role 1 [DE]**	DE-03, DE-04 (FR-DE-03)	Re-executing identical batch creates 0 duplicate target keys via Upsert.
KPI-06	Batch Run Duration <= 60 Seconds	**Role 4 [OB]**	DE-05, OR-06, OB-01 (NFR-Performance)	Live execution duration gauge reflects <= 60s for standard benchmark batch.
KPI-07	100% Run Outcome Logging in DB	**Role 3 [OR]**	OR-06, OB-02 (FR-OR-03)	Every pipeline run produces SUCCESS or FAILED in pipeline_runs table.
KPI-08	4/4 Operational Alert Scenarios	**Role 4 [OB]**	OB-07, QA-06 (FR-OB-04)	Failed, Stale, Never-Reported, and High-Rejection alerts demonstrated.
KPI-09	>= 90% Requirement Traceability	**Role 5 [QA]**	QA-07, DO-01, DO-03 (FR-QA-04)	100% critical requirements and >= 90% total mapped to code and tests.
KPI-10	Operational Runbook Verified	**Role 6 [DO]**	DO-04, DO-05 (FR-DO-05)	Step-by-step walkthrough successfully executed from clean environment.